

Performance Optimization Plan

Current baseline: 100 providers × 20 docs × 34 fields = ~2 hours

Tier 1: Config Changes Only (No Code)

Combined TIER 1 impact: 2 hours → ~1 hour.

#	CHANGE	CURRENT	RECOMMENDED	TIME SAVED	RISK / TRADE-OFF	FILE
1	Increase workers	15	20	~20%	Need 5M TPM	<code>config.json</code> <code>parallel_workers</code>
2	Increase TPM quota	500K	5M	Eliminates retries	None (free)	Azure Portal
3	Text mode for text docs	multimodal	text	13s → 2s per call	Lower accuracy for scanned docs	<code>config.json</code> <code>rag.mode</code>
4	Reduce top_k	3	2	~10% smaller prompts	Slightly less context	<code>config.json</code> <code>rag.top_k</code>
5	Disable eval in prod	true	false	~30% faster	No quality scores	<code>config.json</code> <code>evaluation.enabled</code>
6	Disable reranking	true	false	~100ms per search	Slightly worse chunks	<code>config.json</code> <code>rag.reranking</code>
7	Reduce chunk size	3000	2000	Faster indexing	More chunks, more searches	<code>semantic_chunker</code> <code>config</code>
8	Use faster Azure region	Current	East / West US	13s → 8s per call	May need new resource	Azure Portal

Tier 2: Architecture/Technique Changes (Code Changes)

#	TECHNIQUE	HOW IT WORKS	TIME SAVED	EFFORT	TRADE-OFF
1	Field batching	Group 5-6 fields per GPT call instead of 1	~60%	Medium	Slightly less per-field accuracy
2	Provider-level parallel	Process 2-3 providers simultaneously	~2-3x faster	Medium	More memory, index conflicts possible
3	Embedding cache	Cache field query embeddings (same query reused across docs)	~10%	Easy	Memory usage
4	Skip processed providers	Check if provider already has output → skip	Varies	Easy	Need to re-run manually if needed
5	Hybrid RAG mode	Text mode for simple fields (name, email), multimodal only for complex fields	~40%	Medium	Need to split field configs
6	Pre-index all docs first	Index all docs in batch → then extract all fields	~15%	Medium	Two-pass pipeline
7	Async API calls	<code>asyncio</code> instead of <code>ThreadPoolExecutor</code>	~10%	Medium	Code complexity
8	Smaller model for simple fields	GPT-4o-mini for name/email, GPT-5 for complex fields	~30% cost, ~15% speed	Medium	Two deployments
9	Document filtering	Skip irrelevant documents (e.g., photos, blank pages)	~10-20%	Easy	May miss data in unexpected docs
10	Incremental processing	Only process new/changed documents, not all	Varies	Hard	Need change tracking

Tier 3: Advanced Techniques

#	TECHNIQUE	HOW IT WORKS	TIME SAVED	EFFORT	TRADE-OFF
1	Azure Batch API	Submit all fields as batch job, pick up results later	50% cost, async	Easy	24h turnaround
2	Pre-built field embeddings	Store field query embeddings in file, load at startup	~5%	Easy	Update when prompts change

#	TECHNIQUE	HOW IT WORKS	TIME SAVED	EFFORT	TRADE-OFF
3	Document classification	Classify doc type first → only search relevant fields	~30%	Hard	Need classifier model
4	Two-stage extraction	Stage 1: extract easy fields (fast). Stage 2: only missing fields with multimodal	~40%	Hard	Complex orchestration
5	Distributed processing	Multiple machines processing different providers	Linear scaling	Hard	Infrastructure

Recommended Implementation Order

PRIORITY	WHAT	IMPACT	EFFORT
Now	Config: workers 20, TPM 5M, disable eval for bulk	2h → 1.2h	5 min
Next	Field batching (5-6 fields per call)	1.2h → 30 min	2-3 days
Next	Embedding cache + skip processed	30 min → 25 min	1 day
Later	Hybrid RAG mode (text for simple, multimodal for complex)	25 min → 15 min	2 days
Later	Provider-level parallel (2-3 at once)	15 min → 8 min	2 days

Best single change: Field batching

Reduces GPT calls from 34 per document to 6 per document. Biggest speed gain with moderate effort.